

AI Hackathon 2026

プロジェクト発表資料

期間: 7月4日～7月25日（20時間）

チーム: Kyosuke（Infra Lead） + 3名（Frontend/Backend/QA）

発表日: 7月25日（約10分間）

プロジェクト概要

お出かけプラン AI アシスタント「ことこと町」

架空エリア「ことこと町」を舞台に、複数の欲求（「ランチもしたい、映画も見たい、子連れで」）を1回の入力で解決する、ゆったり系お出かけプランナー。

次フェーズ: プラン生成APIを呼び出すチャット入力・プランカード表示UIの実装

実装済みの機能

- ユーザー認証・権限管理 (admin / store_manager / user)
- 店舗管理 (登録・編集・地図配置・写真管理)
- 予約 (空き確認・作成・キャンセル)
- いいね・レビュー、混雑度の報告と分析ダッシュボード
- 混雑通知メール (cron)、エラー管理ダッシュボード
- **Claude API** によるお出かけプラン生成API (`POST /api/plan/generate`)。本プロジェクトの核)

解決したい課題

ユーザー側の課題

- 複数の欲求を満たす店舗・時間帯の組み合わせを手動で探すのは煩雑
- 混雑や営業時間を考慮した「今から行ける」プランは自力では作りにくい

店舗側の課題

- 混雑の偏り（ピーク集中・閑散時間の空席）が収益を不安定にする

アプローチ

- LLM（Claude API）で欲求を自然言語から解析し、店舗DB × 混雑度 × 営業時間から最適ルートを提案
- 架空エリア採用で商標・実在店舗リスクをゼロにし、デモの再現性を完全にコントロール

システム全体構成

```
Frontend (React 18 + TypeScript / Vite + Tailwind)
  | HTTP / REST API
  ▼
Backend (Node.js + Express / Vercel Functions)
  ビジネスロジック + Claude API 統合 (/api/plan)
  | SQL
  ▼
Database (Supabase / PostgreSQL)
  users / stores / reservations / likes / reviews
  crowd_* / email_* / error_logs / store_media
```

- Deploy: Vercel (main pushで自動デプロイ)
- Notion Sync: PRマージ時に GitHub Actions で自動同期

ディレクトリ構成

```
src/          # フロントエンド (React)
├ pages/      7画面 (Login/Dashboard/Stores/Reservations/...)
├ components/ 15コンポーネント (フォーム・パネル・モーダル等)
├ context/    認証状態管理 (AuthContext)
└ lib/        API クライアント層
server/       # Express (auth/users/stores/media)
api/          # Vercel Functions (reservations/crowd/analytics/plan等)
backend/      # api・server が共有するドメインロジック
tests/        # unit / integration / e2e (31本)
docs/         # 要件定義・DB・チーム運用・発表資料
```

Database Schema（主要テーブル）

Table	役割
users / role_permissions	ユーザー・権限（RBAC）
stores / store_media	店舗マスタ・写真
reservations	予約（空き状況・キャンセル）
likes / reviews	いいね・レビュー
crowd_status / crowd_history / crowd_patterns	混雑度
email_notifications / error_logs	通知メール・エラー記録

AI をどのように利用しているか (1) プロダクト内

Claude API によるプラン生成 (POST /api/plan/generate)

- backend/domains/plan/ : claudeClient (API呼び出しを1箇所に集約) / promptBuilder (店舗×混雑度×レビューからプロンプト構築) / scoring (距離感35%・評価25%・混雑度25%・オファー15%) / schema (zod検証)
- 欲求解析・店舗照合・スコアリング・プラン生成を **API呼び出し1回に統合**
- Claudeの応答はzodスキーマで検証。不正なJSONは502エラーとして処理
- テスト: unit + integration 6件で正常系・異常系を検証

AI をどのように利用しているか（2）開発プロセス

- 4つの役割別サブエージェント（Frontend / Backend / QA / Reviewer）で分業
- レビュー専用エージェントによる品質確保（自己レビュー禁止ルール）
- **敵対的アーキテクチャ監査**: 調査エージェント3系統を並列投入し、22件の改善タスクを特定・手順書化

AIエージェント同士の役割分担

エージェント	役割	視点
Frontend Agent	UI/UX実装、React開発	ユーザー体験
Backend Agent	Claude API統合、ロジック	API・ロジック
QA Agent	テスト・品質確認	品質・信頼性
Reviewer Agent	コードレビュー、セキュリティ審査	正確性・セキュリティ

人間（開発者）の役割

- 企画・要件定義 — 機能・仕様の最終判断
- アーキテクチャ設計 — システム構成の決定
- 統合・デプロイ — 各パートの統合と本番展開
- 進捗管理 — タイムラインと品質監督

技術選定理由（1）

Frontend: React 18 + TypeScript + Tailwind CSS

- コンポーネント再利用・状態管理が容易、型安全性でバグ削減、高速スタイリング

Backend: Node.js + Express

- JavaScript統一で学習コスト削減、軽量でClaude API統合が容易

技術選定理由（2）

Database: Supabase (PostgreSQL)

- マネージドPostgreSQLでインフラ管理不要、リアルタイム機能、認証組み込み

Deployment: Vercel + GitHub Actions

- main push時に自動デプロイ、PRごとにPreview環境、Notion自動同期

工夫した点（1）AIエージェント分業

- 4人のチームメンバーが各自担当領域でAIと協力、レビュー専用サブエージェントで品質を自動確保
 - PRレビュー → AI修正 → マージまで人手を最小化
- 💡 効果: 20時間という限られた時間で多くの機能を実装

工夫した点（2）自動化とドキュメント駆動

- **GitHub → Notion自動同期**: PRマージトリガーのGitHub Actionsで進捗を二重管理
 - **ドキュメント駆動**: CLAUDE.mdで開発ルール・アーキテクチャを一元管理し、発表資料を作業のたびに更新
- 💡 **効果**: 進捗の可視化と、常に「今この瞬間に発表できる状態」を維持

工夫した点（3） 敵対的アーキテクチャ監査

- 「AI生成コードだから品質が高い」という前提を捨て、調査エージェント3系統（フロント/バックエンド/テスト・設定）を並列投入しゼロベースで監査
 - 発見: 実行不能なlint、CI不在、型検査の穴、認可ロジックの分裂、447行のデッドコード等
 - 22件の改善タスクを優先度付き実装手順書に整理（`docs/architecture-audit/`）
- 💡 効果: 「動いているように見える」と「品質が担保されている」の乖離を早期に可視化

工夫した点（4） 監査結果の即実行

- **品質ゲート確立:** ESLint導入・デッドコード削除・型検査範囲拡大
(`tsconfig.backend.json`)
 - **セキュリティ修正:** 無効化ユーザーが管理APIを通過できた欠陥を修正、認可ロジックを一本化
 - **API防御:** エラー形式統一・zod入力検証・CORS許可リスト化・グローバルエラーハンドラ
- 💡 **効果:** lint/build/test (18スイート・136テスト) が全てgreenの状態を確立

工夫した点（5）実装バグの発見・修正

- **cronのUTC/JST変換ミス**: 混雑通知が深夜に実行されていた本番バグを発見・修正
- **フロントのデータアクセス一本化**: Supabase直呼び出しを全廃しREST APIに統一（バンドル 459KB→244KB）
- **ルーティング導入**（react-router-dom）: URL直アクセス・リロード・ブラウザバックに対応
- **いいね・レビュー機能の結線**: 実装済みだがUIから未到達だった問題を解消

苦労した点

- **20時間という限られた時間** → AIエージェント活用で並列開発、スコープを絞って対応
- **AIエージェント間の一貫性維持** → CLAUDE.mdで事前にアーキテクチャを定義、レビュー専用エージェントで品質確保
- **実装 ↔ 発表資料の同期** → 継続的ドキュメント更新を必須化し、常に発表可能な状態を維持

今後の展望

Phase 2: AI提案の高度化

- 複数のClaudeモデルを使い分け（高速なHaiku、高精度なOpus）、提案理由の説明付与、ユーザーフィードバック学習

Phase 3～4: スケーリング・社内展開

- バッチ提案・リアルタイム推奨・多言語対応、他部門（イベント管理・研修計画等）への展開

デモで説明する内容

シーン: ログイン → 店舗探索・予約 → 混雑分析 → (AI提案は統合後)

-  ログイン: テストアカウントでrole別に画面を出し分け (URL直リンク・リロード対応)
-  ことこと町の店舗探索: 一覧・詳細・いいね・レビュー・予約 (空き確認～キャンセル)
-  店舗管理者/管理者: 混雑分析ダッシュボード、店舗・ユーザー管理
-  **AIプラン提案** : API実装完了、UI画面は次フェーズでデモに追加予定

まとめ：目標達成

目標	達成度	備考
AIによる提案機能		プラン生成API実装完了・UI画面は次フェーズ
ユーザー認証・ダッシュボード		ログイン・権限管理完了
データベース（複数テーブル）		正規化済みPostgreSQL
自動デプロイ		Vercel + GitHub Actions
ドキュメント完備		継続的に更新中

まとめ：技術的ハイライトと学習成果

技術的ハイライト

- AIエージェント分業／自動化パイプライン（GitHub→Notion→Vercel）／継続的ドキュメント更新

学習成果

- フルスタック開発（Frontend + Backend + DB）の統合
- AIエージェントを開発プロセスに組み込む実践経験
- AIエージェントを含むチーム開発・分業管理

質疑応答

Contact: Kyosuke Yoshizawa

Repository: <https://github.com/kyousuke-yoshizawa/ai-hackason>

発表日: 2026年7月25日

附録: 技術スタック一覧

- **Frontend:** React 18, TypeScript, Tailwind CSS, Vite
- **Backend:** Node.js, Express, Vercel Functions, Claude API
- **Database:** Supabase (PostgreSQL)
- **Deployment:** Vercel, GitHub Actions
- **Testing:** Jest, Playwright, Testing Library
- **Documentation:** Markdown, Marp (presentation)
- **Team Coordination:** Microsoft Teams, Notion